

Module 5

Sequence Modeling & NLP — RNNs to Transformers

Complete Study Material · Theory + Practical + Real-World Pipeline Context

Tutor: Abhishek and team · Supplementary: Krish Naik one-shot videos (NLP, RNN/LSTM, Transformers)

How to read this material

★ = High-priority — interview-frequent and/or used daily in real projects. ★★ = the Transformer: the single most important architecture in modern AI; every GenAI module (8–10) builds on it.

PIPELINE boxes extend our running fintech project: the loan company now has **text data** — complaints, support chats, application notes — and we build the NLP stack for it.

INTERVIEW · **COMMON MISTAKE** · **WORKED EXAMPLE** boxes as before.

Contents

1. Where Module 5 Sits — Text Enters the Pipeline
2. Text Preprocessing & Word Embeddings ★ (Word2Vec, GloVe)
3. RNNs, GRU and LSTM ★ — Modeling Sequences
4. Sequence-to-Sequence & the Road to Attention ★
5. The Transformer Architecture ★★ — Complete Breakdown
6. Encoder Models: BERT Family — DistilBERT, MobileBERT, TinyBERT ★
7. Decoder Models: GPT ★
8. Hands-On: Fine-tune BERT + Build a GPT Chatbot
9. Test-6 & Test-7 Preparation (incl. Transformer-from-scratch guide)
10. Interview Question Bank with Answers

1. Where Module 5 Sits — Text Enters the Pipeline

So far our loan-default model only sees numbers. But most of a company's data is **text**: complaint emails, support chats, loan-purpose descriptions, KYC documents, call transcripts. Module 5 gives you the machinery to turn language into predictions and generation:

Business need at our fintech	NLP task	This module's tool
10,000 complaint emails/month must reach the right team	Text classification	Fine-tuned BERT (§6, §8)
"Is this customer angry? Escalate?"	Sentiment analysis	Embeddings + classifier / BERT
Predict next month's EMI payment behaviour from a payment <i>sequence</i>	Sequence modeling	RNN/GRU/LSTM (§3)
Support content in English → Hindi/Odia/regional languages	Machine translation	Seq2Seq → Transformer (§4-5, Test-7)
24×7 support chatbot answering loan queries	Text generation	GPT-style decoder (§7-8) → upgraded with RAG in Module 9

THE THROUGH-LINE TO GENAI

This module is the hinge of the whole course. RNN → attention → Transformer is the exact historical path that produced ChatGPT. Master §5 and Modules 8-10 (LLMs, RAG, agents) become straightforward applications; skip it and GenAI stays magic. Also note: the **dot product** you learned in Module 1 §5.2 is about to become the attention mechanism — same formula, trillion-dollar application.

2. Text Preprocessing & Word Embeddings ★

2.1 Basic Text Preprocessing

Models eat numbers, not words. Preprocessing standardizes raw text before conversion:

Step	What	Example	Notes
Lowercasing	Uniform case	"EMI Missed" → "emi missed"	Skip when case matters (NER: "Apple" vs "apple")
Tokenization ★	Split into units (tokens)	"loan not approved" → [loan, not, approved]	Modern models use subword tokenization (§5.1) — handles unseen words
Stopword removal	Drop high-frequency low-info words	remove "is, the, a"	Careful: "not" is a stopwords in some lists but flips sentiment! Don't remove blindly for sentiment tasks
Stemming	Chop suffixes by rule	"payments"→"payment", "studies"→"studi"	Fast, crude — output may not be a word (Porter stemmer)
Lemmatization	Dictionary + POS → root form	"better"→"good", "studies"→"study"	Slower, accurate — prefer for quality pipelines
Cleaning	Remove HTML, URLs, punctuation, extra spaces; handle emojis/numbers per task	—	Regex (<code>re</code> module) is your friend

```
import re, nltk
from nltk.corpus import stopwords
from nltk.stem import WordNetLemmatizer

lemm = WordNetLemmatizer(); stops = set(stopwords.words("english")) - {"not", "no"}
def preprocess(text):
    text = re.sub(r"http\S+|[\^a-zA-Z\s]", " ", text.lower()) # strip urls/punct/digits
    tokens = [lemm.lemmatize(w) for w in text.split() if w not in stops]
    return " ".join(tokens)

preprocess("EMI of ₹12,000 NOT credited!! see https://bank.com")
# → "emi not credited see"
```

INTERVIEW ALERT

"Stemming vs lemmatization?" is a guaranteed NLP screening question. Also: "Why can removing stopwords hurt sentiment analysis?" (negations) and "What is tokenization; what is subword tokenization and why?" (§5.1 answer).

2.2 From Words to Numbers — the Evolution

1. **Bag of Words / Count Vectorizer:** vocabulary-sized vector of word counts. Loses order; huge sparse vectors.
2. **TF-IDF:** weight = term frequency × log(N/docs-containing-term) — downweights common words, upweights distinctive ones. Still sparse, still no meaning: "loan" and "credit" are as unrelated as "loan" and "pizza".
3. **Word Embeddings ★:** each word → a **dense** vector (50–300 dims) learned so that **similar words get nearby vectors**. Meaning becomes geometry.

THE CORE IDEA (DISTRIBUTIONAL HYPOTHESIS)

"You shall know a word by the company it keeps." Words appearing in similar contexts (EMI / installment / repayment) end up with similar vectors. Similarity = cosine similarity = the normalized dot product from Module 1. The famous demo: $\text{vector}(\text{"king"}) - \text{vector}(\text{"man"}) + \text{vector}(\text{"woman"}) \approx \text{vector}(\text{"queen"})$ — embeddings capture relationships as directions.

2.3 Word2Vec ★

- A shallow neural network trained on a fake task so that the *weights* become the embeddings. Two flavours:
- **CBOW** (Continuous Bag of Words): predict the center word from its context words. Faster; good for frequent words.
- **Skip-gram**: predict context words from the center word. Slower; better for rare words and small corpora.
- Training tricks: **negative sampling** (instead of a full-vocabulary softmax, learn to distinguish true context pairs from a few random "negative" words) — makes training feasible.

```
from gensim.models import Word2Vec
sents = [["emi", "payment", "delayed"], ["loan", "approved", "quickly"], ...]
w2v = Word2Vec(sents, vector_size=100, window=5, sg=1, min_count=2) # sg=1 → skip-gram
w2v.wv.most_similar("emi") # → [("installment", 0.81), ("payment", 0.77), ...]
w2v.wv["loan"] # the 100-dim vector itself
```

2.4 GloVe

- **Global Vectors**: instead of sliding a window (local, like Word2Vec), GloVe factorizes the global **word co-occurrence matrix** of the whole corpus — dot products of vectors are trained to match log co-occurrence counts.
- In practice: pre-trained GloVe vectors (glove.6B, trained on Wikipedia) are downloaded and used as ready-made embeddings — a common baseline and a frequent interview mention.

THE LIMITATION THAT MOTIVATES EVERYTHING AFTER

Word2Vec/GloVe give ONE fixed vector per word. But "bank" in "river bank" vs "HDFC bank" means different things — **static embeddings cannot capture context**. Contextual embeddings (ELMo → BERT, §6) fix exactly this: the vector for "bank" depends on the sentence around it. Interviewers love asking "limitation of Word2Vec?" — this is the answer, plus out-of-vocabulary words (fixed vocabulary).

3. RNNs, GRU and LSTM ★

3.1 Why Feed-Forward Networks Fail on Sequences

An FFNN (Module 4) takes a fixed-size input and treats features independently. Language is a **sequence**: variable length, and order matters ("loan approved not" $\neq$ "loan not approved"). We need a network with **memory**.

3.2 Simple RNN

$$h_t = \tanh(W_{xh} x_t + W_{hh} h_{t-1} + b) \quad \cdot \quad y_t = W_{hy} h_t$$

- At each time step the RNN reads the current token x_t and its previous **hidden state** h_{t-1} (its memory), producing a new hidden state. The **same weights are reused** at every step (parameter sharing) — that's how one network handles any sequence length.
- Training = **Backpropagation Through Time (BPTT)**: unroll the network across time steps and backpropagate as usual.

THE FATAL FLAW — VANISHING/EXPLODING GRADIENTS ★

BPTT multiplies gradients through W_{hh} at every step. Over 50+ steps, values <1 shrink to ~ 0 (**vanishing** — the network forgets long-range context: word 3 cannot influence word 80) and values >1 blow up (**exploding** — fixed by gradient clipping). This is Module 4's vanishing-gradient problem, amplified by depth-in-time. It is THE reason LSTM/GRU exist — state this causal chain in interviews.

3.3 LSTM — Long Short-Term Memory ★

LSTM adds a **cell state** C_t — a conveyor belt of memory flowing through time with only minor, *gated* edits. Three gates (each a sigmoid layer outputting 0-1 = "how much to let through"):

Gate	Question it answers	Formula (concept)
Forget gate f_t	What old memory should we erase?	$\sigma(W_f \cdot [h_{t-1}, x_t])$
Input gate i_t	What new information should we store?	$\sigma(W_i \cdot [h_{t-1}, x_t]) \times \tanh \text{ candidate } \tilde{C}_t$
Output gate o_t	What part of memory do we reveal as h_t ?	$o_t \times \tanh(C_t)$

$$C_t = f_t \odot C_{t-1} + i_t \odot \tilde{C}_t \quad \text{— memory update is **additive**, so gradients flow along C almost undiminished} \rightarrow \text{long-range memory works}$$

3.4 GRU — Gated Recurrent Unit

- A streamlined LSTM: merges cell & hidden state, and uses two gates — **update gate** (how much old vs new) and **reset gate** (how much past to consult for the new candidate).
- Fewer parameters $\rightarrow$ faster training, less overfitting on small data; LSTM can edge ahead on very long sequences. In practice: try GRU first on small/medium data — a good interview answer.

3.5 Comparison + Code

	Simple RNN	GRU	LSTM
Gates	0	2 (update, reset)	3 (forget, input, output) + cell state
Long-range memory	Poor (vanishing grad.)	Good	Good-best
Parameters / speed	Fewest / fastest	Middle	Most / slowest
Use today	Teaching only	Small/medium sequence tasks	Time series, legacy NLP; largely replaced by Transformers for text

```
# Keras: compare RNN / GRU / LSTM on complaint sentiment (Activity + Test-6 skeleton)
import tensorflow as tf
from tensorflow.keras import layers, Sequential

def make_model(cell):
    return Sequential([
        layers.Embedding(input_dim=20000, output_dim=128), # learn embeddings end-to-end
        cell(64), # layers.SimpleRNN / GRU / LSTM
        layers.Dense(1, activation="sigmoid")])

for cell in [layers.SimpleRNN, layers.GRU, layers.LSTM]:
    m = make_model(cell)
    m.compile("adam", "binary_crossentropy", metrics=["accuracy"])
    m.fit(X_train, y_train, validation_split=0.2, epochs=5, batch_size=64)
# Expected finding: RNN plateaus lowest; GRU ≈ LSTM, GRU trains faster. Report curves side by side.
# Bidirectional wrapper reads the sequence both ways: layers.Bidirectional(layers.LSTM(64))
```

PIPELINE CONNECTION

Beyond text: a customer's last 12 months of EMI payments is a numeric *sequence*. An LSTM over payment history predicts next-month delinquency better than static features alone — a real feature-upgrade to our Module 2 default model, and a classic fintech use of sequence models.

4. Sequence-to-Sequence & the Road to Attention ★

4.1 Encoder-Decoder (Seq2Seq)

Translation maps a sequence to a *different-length* sequence: "How are you?" (3 tokens) → "आप कैसे हैं?" (3) or Odia equivalent. Architecture:

- **Encoder** (an LSTM/GRU): reads the source sentence, compresses it into a final hidden state — the **context vector**.
- **Decoder** (another LSTM/GRU): starts from the context vector and generates the target one token at a time, feeding each output back as the next input, until it emits <EOS>.
- **Teacher forcing** (training trick ★): during training, feed the decoder the *true* previous target token instead of its own (possibly wrong) prediction — stabilizes and speeds up learning. At inference you must use the model's own outputs (this train/inference mismatch is called exposure bias — nice interview depth point).

THE BOTTLENECK PROBLEM

The entire source sentence — 5 words or 50 — must squeeze through ONE fixed-size context vector. Long sentences degrade badly: by the end of encoding, the beginning is half-forgotten. Measured reality: Seq2Seq BLEU scores drop sharply beyond ~30 words.

4.2 Attention — the Fix ★

CORE IDEA

Instead of one summary vector, let the decoder **look back at ALL encoder hidden states at every step**, and learn *where to look*. Generating the Odia word for "loan"? Attend mostly to the encoder state of "loan". Attention weights = a softmax over relevance scores; the context for each decoding step = weighted sum of all encoder states. Relevance score = (surprise!) a **dot product** between decoder state and encoder states — Module 1 §5.2 again.

- Bonus: attention weights are **interpretable** — plot them as a heatmap and you literally see a soft word-alignment between source and target.
- **Self-attention**: apply the same trick *within* one sentence — every word attends to every other word to build its representation. "The customer closed his account because **it** had charges" → "it" attends to "account". This is the single mechanism the Transformer is built from.

5. The Transformer Architecture ★★

2017, "Attention Is All You Need": drop recurrence entirely, use only self-attention + feed-forward layers. Two consequences changed AI: (1) all tokens process **in parallel** (RNNs are sequential → slow), enabling training on internet-scale data; (2) every token connects to every other in one hop → long-range dependencies handled perfectly. GPT, BERT, Llama, Gemini — all are Transformers. Learn this section cold.

5.1 Input: Tokenization + Embeddings + Positional Encoding

- **Subword tokenization** (BPE / WordPiece): split rare words into pieces ("unaffordable" → un + afford + able). Fixed vocab (~30–50K) yet no out-of-vocabulary failures — fixes Word2Vec's OOV problem.
- **Token embedding**: lookup table token-id → d-dimensional vector (d = 512 in the paper; thousands in modern LLMs). Learned during training.
- **Positional encoding** ★: self-attention is order-blind (a set operation) — "dog bites man" = "man bites dog" without it! Add a position-dependent vector to each embedding. Original: fixed sinusoids of different frequencies, $PE(pos, 2i) = \sin(pos/10000^{2i/d})$, $PE(pos, 2i+1) = \cos(\cdot)$ — lets the model reason about relative positions. Modern models often learn positions or use RoPE; the "why" is what interviews test.

5.2 Self-Attention: Query, Key, Value ★★ (THE interview question of the decade)

Each token's embedding x is projected into three vectors by learned matrices: **Query** = xW_Q ("what am I looking for?"), **Key** = xW_K ("what do I offer?"), **Value** = xW_V ("what information do I carry?").

$$\text{Attention}(Q, K, V) = \text{softmax}(QK^T / \sqrt{d_k}) V$$

1. **QK^T** — dot product of every query with every key → an $n \times n$ matrix of raw relevance scores (n = sequence length).
2. **$\div \sqrt{d_k}$** — scaling. Without it, large d_k makes dot products huge → softmax saturates → gradients vanish. ("Why divide by $\sqrt{d_k}$?" is a favourite follow-up — this is the answer.)
3. **softmax** per row → attention weights (each row sums to 1: how much token i attends to each token j).
4. **$\times V$** — each token's new representation = weighted average of all tokens' values. Context flows in.

WORKED MICRO-EXAMPLE

Sentence: "emi not paid". Token "paid" computes $q_{\text{paid}} \cdot k_{\text{emi}}$, $q_{\text{paid}} \cdot k_{\text{not}}$, $q_{\text{paid}} \cdot k_{\text{paid}}$ → say scores [2.1, 3.4, 1.0] → softmax → weights [0.24, 0.66, 0.10] → new "paid" vector = $0.24 \cdot v_{\text{emi}} + 0.66 \cdot v_{\text{not}} + 0.10 \cdot v_{\text{paid}}$. The negation "not" now lives inside "paid"'s representation — meaning became contextual. Complexity: $O(n^2 \cdot d)$ in sequence length — the reason long-context LLMs are expensive (interview depth point).

5.3 Multi-Head Attention ★

- Run h parallel attention "heads" (8 in the paper), each with its own W_Q , W_K , W_V projecting to d/h dimensions; concatenate outputs and project with W_O .
- **Why**: different heads learn different relationship types simultaneously — one tracks syntax (subject↔verb), another coreference (it↔account), another proximity. One head would have to average all of these.

5.4 The Rest of the Encoder Block

- **Position-wise Feed-Forward Network (FFN)**: two linear layers with activation, applied to each token independently: $FFN(x) = W_2 \cdot \text{act}(W_1 x)$. Typically expands $d \rightarrow 4d \rightarrow d$; where much of the model's "knowledge capacity" lives.

- **Residual connections:** $\text{output} = x + \text{Sublayer}(x)$ — gradients get a highway past each block (Module 4/6's ResNet idea), enabling very deep stacks.
- **Layer Normalization:** normalize across the feature dimension per token (not across the batch like BatchNorm — sequence lengths vary, batches are awkward). Stabilizes training. "LayerNorm vs BatchNorm and why LN in Transformers" — a standard interview question; this is the answer.
- Stack N such blocks (6 in the paper; 30–100+ in modern LLMs).

5.5 The Decoder Block — Two Changes

- **Masked multi-head self-attention ★:** when generating, token t must not see tokens $t+1...$ (they don't exist yet at inference, and peeking during training = cheating). Implementation: set future positions' scores to $-\infty$ before softmax $\rightarrow$ their weights become 0. This "causal mask" is what makes GPT autoregressive.
- **Encoder-decoder (cross-)attention:** the decoder's Queries attend over the encoder output's Keys/Values — this is where the target side reads the source sentence (§4.2's attention, now inside the Transformer).
- **Output head:** final linear layer projects each position to vocabulary size $\rightarrow$ **softmax** $\rightarrow$ probability distribution over the next token. Training loss: cross-entropy vs the true next token, with teacher forcing.

5.6 One-Page Mental Map (reproduce this from memory before Test-7)

Stage	Encoder (understanding)	Decoder (generation)
Input	Source tokens $\rightarrow$ embedding + positional encoding	Target-so-far tokens $\rightarrow$ embedding + positional encoding
Attention 1	Self-attention (bidirectional — every token sees all)	Masked self-attention (each token sees only the past)
Attention 2	—	Cross-attention over encoder output (Q from decoder, K/V from encoder)
Then	FFN $\rightarrow$ (each sublayer wrapped in residual + LayerNorm)	FFN $\rightarrow$ residual + LayerNorm $\rightarrow$ after N blocks: Linear $\rightarrow$ Softmax $\rightarrow$ next-token probabilities
Used alone as...	Encoder-only = BERT family (§6): understanding tasks	Decoder-only = GPT family (§7): generation. Full pair = translation (T5, our Test-7)

INTERVIEW ALERT — THE BIG FIVE

1) "Explain self-attention / Q,K,V" ★★ (know the formula + the intuition + the worked example). 2) "Why positional encoding?" 3) "Why $\sqrt{d_k}$ scaling?" 4) "Purpose of masking in the decoder?" 5) "Why did Transformers replace RNNs?" (parallelism + $O(1)$ -hop long-range dependencies vs sequential $O(n)$ paths). If you can whiteboard §5.6's table, you clear most NLP rounds.

6. Encoder Models: BERT Family ★

6.1 BERT — the Foundation

- **BERT** = Bidirectional Encoder Representations from Transformers: the Transformer *encoder* stack only, pre-trained on massive text, producing **contextual embeddings** (fixing Word2Vec's one-vector-per-word limitation from §2.4).
- **Pre-training tasks:** (1) **Masked Language Modeling (MLM)** ★ — randomly mask 15% of tokens, predict them from BOTH directions ("The [MASK] was not credited" → "EMI"); (2) Next Sentence Prediction (NSP) — does sentence B follow A? (later work found NSP weakly useful).
- **Fine-tuning paradigm** ★: take pre-trained BERT + add a small task head (classifier on the [CLS] token) + train briefly on your labeled data. Pre-train once on the internet, fine-tune cheaply per task — this two-phase recipe is the template all of modern NLP follows, and the essence of transfer learning for text.

6.2 The Distilled Family — DistilBERT, MobileBERT, TinyBERT

BERT-base = 110M parameters — heavy for production latency budgets and mobile. **Knowledge distillation** ★: train a small "student" model to mimic a large "teacher's" output probabilities (soft labels carry richer signal than hard labels: teacher says "complaint: 0.7 billing, 0.25 service" — the student learns the relationship, not just the top answer).

Model	Idea	Size vs BERT-base	Speed	Quality retained	Use when
DistilBERT	General distillation during pre-training; 6 layers vs 12	~40% smaller (66M)	~60% faster	~97% of GLUE score	Default production choice for classification — our complaint-triage model (§8)
MobileBERT	Thin, deep bottleneck architecture designed for phones; distilled from a special teacher	~4.3× smaller (25M)	Optimized for mobile latency	Near BERT-base on many tasks	On-device NLP: keyboards, offline apps
TinyBERT	Two-stage distillation: mimic teacher's <i>attention maps & hidden states</i> too, plus task-specific distillation	~7.5× smaller (14.5M)	~9.4× faster	~96% of teacher on GLUE	Extreme latency/edge constraints

INTERVIEW ALERT

"What is knowledge distillation?" and "How would you deploy a BERT-quality model under a 10 ms latency budget?" (→ distilled model, quantization [Module 9], ONNX/batching). Also: "BERT vs GPT difference?" — answered fully in §7.3.

7. Decoder Models: GPT ★

7.1 The Autoregressive Recipe

- **GPT** = Generative Pre-trained Transformer: the Transformer *decoder* stack only (masked self-attention; no encoder, no cross-attention).
- **Pre-training task — Causal Language Modeling (CLM) ★**: just predict the next token, over trillions of tokens. Deceptively simple; at scale it forces the model to learn grammar, facts, reasoning patterns — because all of them help predict what comes next.
- **Generation loop**: prompt → predict next-token distribution → pick a token (greedy / temperature sampling / top-p — full treatment in Module 8) → append → repeat. Each new token attends to everything before it.

7.2 GPT Evolution in One Table

Model	Year	Scale (approx.)	What it proved
GPT-1	2018	117M	Pre-train + fine-tune works for decoders
GPT-2	2019	1.5B	Zero-shot: one model, many tasks, no fine-tuning
GPT-3	2020	175B	In-context / few-shot learning ★ : show examples in the prompt, model adapts — no weight updates. The birth of prompt engineering (Module 8)
InstructGPT/ ChatGPT	2022	—	RLHF alignment: instruction-following + human preference tuning turned a text predictor into an assistant
GPT-4 / modern LLMs	2023+	undisclosed	Multimodal, long context; same decoder-only core as GPT-1

7.3 BERT vs GPT — the Comparison You Must Nail ★

	BERT (encoder-only)	GPT (decoder-only)
Attention	Bidirectional — sees whole sentence	Causal/masked — sees only the past
Pre-training	Masked LM (fill in blanks)	Next-token prediction
Natural strength	Understanding : classification, NER, extractive QA, embeddings/retrieval	Generation : chat, summarization, translation, code
Output	A vector per token (feed to a head)	Text, token by token
In our pipeline	Complaint triage, semantic search embeddings (→ RAG retriever, M9)	Support chatbot (→ RAG generator, M9)

8. Hands-On: Fine-tune BERT + Build a GPT Chatbot

8.1 Fine-tune DistilBERT for Complaint Classification (Activity)

```
from transformers import (AutoTokenizer, AutoModelForSequenceClassification,
                          TrainingArguments, Trainer)
from datasets import load_dataset
import numpy as np, evaluate

# Our fintech scenario: complaints → {billing, loan_processing, app_issue, fraud}
ds = load_dataset("csv", data_files="complaints.csv") # columns: text, label
tok = AutoTokenizer.from_pretrained("distilbert-base-uncased")
ds = ds.map(lambda b: tok(b["text"], truncation=True, padding="max_length",
                          max_length=128), batched=True)
ds = ds["train"].train_test_split(test_size=0.2, seed=42)

model = AutoModelForSequenceClassification.from_pretrained(
    "distilbert-base-uncased", num_labels=4)

f1 = evaluate.load("f1")
def metrics(p):
    preds = np.argmax(p.predictions, axis=1)
    return f1.compute(predictions=preds, references=p.label_ids, average="macro")

args = TrainingArguments("cmlnt-model", num_train_epochs=3,
                          per_device_train_batch_size=16, learning_rate=2e-5, # small LR: don't wreck ★
                          eval_strategy="epoch", weight_decay=0.01) # pre-trained knowledge
trainer = Trainer(model, args, train_dataset=ds["train"],
                  eval_dataset=ds["test"], compute_metrics=metrics)
trainer.train()
trainer.save_model("complaint-classifier") # → serve behind FastAPI exactly as Module 2 §8!
```

FINE-TUNING GOTCHAS (TEST-6 RELEVANT)

1) Learning rate too high (e.g., 1e-3) destroys pre-trained weights — "catastrophic forgetting"; stay in the 1e-5...5e-5 band. 2) Use macro-F1 on imbalanced complaint classes, not accuracy (Module 2 lesson repeats). 3) 3-4 epochs is usually enough — more = overfitting on small data. 4) Match the tokenizer to the checkpoint — mixing tokenizers silently corrupts inputs.

8.2 GPT-based Chatbot (Activity)

```
# Option A – local/open-source (no API cost): DialoGPT via HF pipeline
from transformers import pipeline, Conversation
chat = pipeline("conversational", model="microsoft/DialoGPT-medium")
conv = Conversation("What documents do I need for a personal loan?")
print(chat(conv))

# Option B – production pattern: OpenAI-style API + a system prompt + memory
from openai import OpenAI
client = OpenAI()
history = [{"role": "system", "content":
    "You are LoanBuddy, a polite support assistant for a lending company. "
    "Answer only loan-related queries; escalate anything about fraud to a human."}]
def ask(user_msg):
    history.append({"role": "user", "content": user_msg})
    r = client.chat.completions.create(model="gpt-4o-mini", messages=history)
    reply = r.choices[0].message.content
    history.append({"role": "assistant", "content": reply}) # memory = resend history
    return reply
# Wrap `ask` in Streamlit (st.chat_input / st.chat_message) → demo UI in 20 lines.
# Limitation you'll hit: the bot doesn't know YOUR loan policies → hallucinates.
# The fix is retrieval over your documents = RAG – exactly Module 9. Remember this pain.
```

9. Test-6 & Test-7 Preparation

Test-6 — Recurrent / transfer-learning model + performance enhancement

- Baseline first (simple LSTM or DistilBERT), record metric → then *enhance* and show the delta. Enhancement menu: Bidirectional LSTM, stacked layers, dropout & recurrent_dropout, pre-trained GloVe embeddings in the Embedding layer (vs random init), learning-rate scheduling, early stopping, class weights, longer max_length, or swap LSTM→DistilBERT fine-tune. Examiners reward the **comparison table + reasoning**, not just the final number.

Test-7 — Transformer from scratch: English → Odia translation

1. **Data prep:** parallel sentence pairs; add <SOS>/<EOS>; build subword tokenizers per language (or a shared one); pad + attention masks.
2. **Build exactly §5.6:** embedding + positional encoding → N encoder blocks (self-attn → FFN, residual+LN) → N decoder blocks (masked self-attn → cross-attn → FFN) → linear+softmax. Start small: d=128, heads=4, N=2 — it must fit class compute.
3. **Train:** teacher forcing, cross-entropy ignoring pad tokens, Adam with warmup; watch for the causal-mask bug (if val loss is suspiciously perfect, your decoder is peeking!).
4. **Inference:** greedy decode loop (feed outputs back until <EOS>); beam search = bonus marks.
5. **Evaluate:** BLEU score (n-gram overlap with reference translation) + show 5 sample translations, including one long sentence to discuss limitations.
6. Odia-specific notes: use a Unicode-clean corpus (e.g., Samanantar / OPUS pairs), normalize Odia Unicode, and expect subword tokenization to matter a lot for a morphologically rich Indic language — mention this in your write-up for extra credit.

10. Interview Question Bank — Module 5

Q1. Stemming vs lemmatization?

Stemming: rule-based suffix chopping, fast, may produce non-words ("studi"). Lemmatization: dictionary+POS to true root ("better"→"good"), slower, accurate.

Q2. TF-IDF vs word embeddings? ★

TF-IDF: sparse, vocabulary-sized, frequency-weighted, no semantics (synonyms unrelated).
Embeddings: dense low-dim vectors where similar meanings are geometrically close; learned from context.

Q3. CBOW vs Skip-gram?

CBOW predicts center word from context (fast, frequent words); Skip-gram predicts context from center (slower, better for rare words/small corpora). Negative sampling makes both trainable at scale.

Q4. Limitations of Word2Vec/GloVe? ★

One static vector per word — no context ("bank" river vs finance); OOV failures with fixed vocab; no word-order awareness. Fixed by contextual embeddings (BERT) + subword tokenization.

Q5. Why do RNNs suffer vanishing gradients, and how does LSTM fix it? ★

BPTT multiplies through W repeatedly → gradients shrink/explode over long spans. LSTM's cell state updates additively through a forget-gated highway, so gradients flow across many steps; gates learn what to keep/write/reveal.

Q6. GRU vs LSTM — when which?

GRU: 2 gates, fewer params, faster, strong default on small/medium data. LSTM: 3 gates + cell state, slight edge on very long dependencies. Empirically task-dependent — say you'd A/B them.

Q7. What is teacher forcing? Its drawback?

Training decoder on ground-truth previous tokens instead of its own predictions → faster, stabler training. Drawback: exposure bias — at inference the model must consume its own (imperfect) outputs, a distribution it never saw in training.

Q8. What problem does attention solve in Seq2Seq? ★

The fixed-size context-vector bottleneck: long inputs get lossy. Attention lets each decoding step take a learned weighted view over ALL encoder states — quality no longer decays with length, and weights give interpretable alignment.

Q9. Explain self-attention with Q, K, V. ★★

Each token projects to Query (what I seek), Key (what I offer), Value (my content). $\text{Scores} = QK^T/\sqrt{d_k} \rightarrow \text{softmax} \rightarrow \text{weights} \rightarrow \text{output} = \text{weighted sum of Values}$. Every token's new representation mixes in relevant context from the whole sequence; computed for all tokens in parallel.

Q10. Why divide by $\sqrt{d_k}$?

With large d_k , dot products have large variance → softmax saturates at near-one-hot → tiny gradients. Scaling keeps scores in a trainable range.

Q11. Why positional encoding? ★

Self-attention is permutation-invariant — without position info, word order is invisible. Sinusoidal (or learned/RoPE) position vectors added to embeddings restore order awareness.

Q12. Purpose of multi-head attention?

Parallel heads in different subspaces capture different relation types simultaneously (syntax, coreference, position); concatenation + projection combines them — richer than one averaged attention pattern.

Q13. What does the decoder's mask do? ★

Sets future positions to $-\infty$ pre-softmax so token t attends only to $\leq t$. Preserves the autoregressive property: no peeking at answers during training; matches generation-time reality.

Q14. Why did Transformers replace RNNs? ★

(1) Full parallelism across sequence positions → internet-scale training feasible (RNNs are inherently sequential). (2) Any-to-any attention gives $O(1)$ path length for long-range dependencies vs $O(n)$ in RNNs. Cost: $O(n^2)$ attention in sequence length.

Q15. LayerNorm vs BatchNorm — why LN in Transformers?

BatchNorm normalizes per-feature across the batch — unstable with variable-length sequences/small batches and couples examples. LayerNorm normalizes across features per token, batch-independent — fits sequence models.

Q16. BERT vs GPT? ★

BERT: encoder-only, bidirectional, masked-LM pre-training → understanding tasks (classification/NER/QA/embeddings). GPT: decoder-only, causal attention, next-token pre-training → generation. Full encoder-decoder (T5) → translation/summarization.

Q17. What is masked language modeling?

Randomly mask ~15% of tokens; model predicts them using both left and right context — the pre-training objective that gives BERT bidirectional contextual representations.

Q18. What is knowledge distillation? Name distilled BERTs. ★

Train a small student to match a large teacher's soft output distributions (and optionally hidden states/attention maps) — retains most quality at a fraction of size/latency. DistilBERT (~40% smaller, ~97% quality), MobileBERT (mobile-optimized), TinyBERT (two-stage distillation, ~9x faster).

Q19. What is in-context / few-shot learning?

GPT-3's emergent ability: task examples placed in the prompt steer the model without any weight updates — the foundation of prompt engineering (Module 8).

Q20. How would you build a text classifier today, start to finish?

Baseline TF-IDF + LogisticRegression for a floor → fine-tune DistilBERT (tokenize, small LR $2e-5$, 3 epochs, macro-F1, stratified split) → error analysis → serve the saved model behind FastAPI in Docker (Module 2 §8) → monitor drift. Mentioning the full lifecycle, not just the model, is what separates candidates.

RESOURCES FOR MODULE 5

- Krish Naik — NLP playlist one-shots (Word2Vec, RNN/LSTM/GRU, Transformers, BERT) (assigned).
- "The Illustrated Transformer" & "The Illustrated BERT" — Jay Alammar (the best visual explanations in existence; read before Test-7).
- *Hands-On Large Language Models* — O'Reilly (course book), ch. on tokens/embeddings & Transformers.
- Hugging Face NLP Course (free) — chapters 1-3 map exactly to §6-8.
- 3Blue1Brown — "Attention in Transformers" video for §5 intuition.